

Adaptive Traffic Control using Deep Learning and Arduino Simulation

Riya Kumari¹[0000-1111-2222-3333], Sneha Suman¹[1111-2222-3333-4444] and Jasdeep Kaur
Dhanoo¹[2222-3333-4444-5555]

¹ Department of Electronics and Communication Engineering,
Indira Gandhi Delhi Technical University for Women, Delhi, India
kriya9464@gmail.com
snehasuman810@gmail.com
jasdeep@igdtuw.ac.in

Abstract. An efficient and intelligent traffic control system is essential for reducing unnecessary congestion and fuel wastage. This study presents a smart traffic control system that dynamically adjusts green light duration according to real-time traffic density. Three object detection models - YOLOv5, Faster R-CNN, and SSD - were implemented and evaluated on a traffic image dataset from Kaggle. YOLOv5 showed superior performance with the lowest inference times (72.59s for training, 3.21s for testing, and 6.80s for validation), outperforming Faster RCNN and SSD. Due to its speed, YOLOv5 was selected for further processing. The vehicle count detected by this approach was transmitted to an Arduino-based system simulated in SimulIDE, through serial communication. This approach focuses on reducing the wait time and enhancing traffic flow efficiency.

Keywords: Traffic Management, YOLO, Traffic density, Arduino, Green time.

1 Introduction

In the current traffic system, traffic lights operate at fixed intervals. This approach is efficient when there is heavy traffic that is all the lanes have heavy vehicle density. Fixed time interval ensures equal waiting time and efficient traffic management. However, this approach becomes ineffective in low-traffic situations. For example, one lane has very little or no traffic but the red light is on for a fixed time interval that is not required and the opposite lane has heavy traffic then in this condition, they have to wait unnecessarily which leads to time wastage. To tackle this problem authors have proposed a solution to calculate green light duration based on dynamic traffic conditions.

This proposed solution leverages advanced models such as YOLOv5 launched on GitHub, Faster R-CNN by Ren et al. [1], and SSD by Liu et al. [2] which calculate the vehicle density and dynamically adjust traffic light timing. These models analyze the traffic images captured by cameras installed near the traffic lights. So by estimating the number of vehicles in the lane, the algorithm will take and adjust the green light duration accordingly. A recent study by Khan et al. [3] also attempted to couple object detection and adaptive logic, to underline the interest in vision-driven signal timing.

The traffic image dataset which is used in this study, comprises the images that are captured by cameras installed near traffic lights. The deep learning models are evaluated on the dataset and compared based on their timing. YOLOv5 demonstrating the lowest processing time is selected for further steps. The calculated vehicle count was transmitted to a virtual Arduino circuit in SimulIDE via serial communication, where the Arduino shows the best duration for the green light and activates the light. This approach aims to reduce delays, improve efficiency, and minimize fuel wastage. To tackle these, our proposed work makes the following contributions:

- Comparative analysis of YOLOv5, Faster R-CNN, and SSD for traffic density estimation.
- Integration of the best-performing model (YOLOv5) with an Arduino-based traffic light simulator.
- Development of an adaptive signal timing formula based on vehicle count.
- End-to-end simulation of an adaptive traffic control system using SimulIDE.

2 Related Work

The current traffic management system heavily depends on the preset timers which operate on fixed timers and display green and red lights alternatively for every cycle without considering the current traffic congestion. This calls for the need to have an adaptive traffic system in order to make the system more efficient. To address this, a study by Shingate et al. [4] gives an idea of how to implement reinforcement learning, which is a branch of machine learning having two primary elements - Agent and Environment. This method shows an optimal policy based on reward feedback.

Tomar et al. [5] provided a comprehensive review of traffic signal control strategies, including traditional fixed-time systems, sensor-based, vision-based, and learning-based algorithms. Their study also explores optimization techniques like genetic algorithms and swarm optimization, also discussed by Khasawneh and Awasthi [6], as well as connected vehicle systems and fuzzy logic. These methods aim to improve traffic flow and reduce congestion, with the paper highlighting the potential of combining these approaches for more effective solutions in real-world traffic management.

A comparative study by Rahman et al. [7] using the Dhaka traffic image dataset, evaluated several models such as RCNN, Fast RCNN, Faster RCNN, and YOLOv5. Their results show that YOLOv5 surpasses the other models, achieving a mean average precision (mAP@0.5) of 0.458 with an inference time of 0.75 seconds. It highlights YOLOv5's effectiveness in detecting traffic in densely populated areas, utilizing Non-Maximum Suppression Ensembling to improve accuracy and real-time detection capabilities.

Another analysis by Mehta et al. [8] explores the RCNN, Fast RCNN, Faster RCNN, SSD, YOLOv4, and YOLOv8 object detection models, and compares them on their efficiencies to detect a vehicle and traffic predictions under different cases. However YOLOv4 works well, YOLOv8 also shows impressive accuracy and performs at par with

the YOLOv4. However, the YOLO family performs well with respect to the RCNN family.

A study by Chaudhuri [9] solely focuses on the Faster RCNN algorithm for the segmentation of vehicles on the COCO and DAWN datasets. This architecture utilizes adaptive background modeling and also tackles the illumination and shadow artifacts. It caters to this by following steps such as reduction with an adaptive background model, then Faster RCNN based subnet operation, followed by initial refinement using Faster RCNN, and lastly the result is optimized by extended topological active nets.

Beyond the vision-based, sensor-integrated systems have also shown some value. Jagadeesh et al. [10] introduce an intelligent traffic management system that dynamically adjusts traffic light sequences based on live traffic density data acquired via sensors. The sensor data is processed by low-power embedded controllers, which analyze the vehicle count and determine the optimal duration for green lights. This is done by implementing comparators, low-power embedded controllers, storage devices, and IR sensors.

Tabassum et al. [11] proposed a system to integrate object detection using YOLO on CCTV images to estimate traffic density and prioritize emergency vehicles. The system consists of two key components: a traditional traffic management system and an ambulance detection module, both managed by a centralized Arduino Mega 2560 microcontroller. By using Python OpenCV, the system processes real-time traffic images, identifying congestion patterns and dynamically controlling traffic signals to optimize flow while ensuring rapid passage for emergency vehicles.

Srivastava et al. [12] presented an alternative sensor-based method that uses PIR (Proximity Infrared Sensors) to detect vehicle presence and measure traffic congestion. This data is processed by an Arduino microcontroller, which determines the duration of green lights for each lane accordingly. Sensors placed along the road continuously monitor vehicle movement, transmitting data to the controller, which decides signal switching to optimize traffic flow.

Finally, Gupta et al. [13] propose a traffic management system that integrates ultrasonic sensors and gas sensors to adjust traffic signal timings dynamically, based on real-time traffic density and air pollution levels. The system is built using an Arduino Uno R3, which processes data from the ultrasonic sensor to measure vehicle density and the gas sensor to monitor air quality. The traffic lights (red, yellow, and green LEDs) are controlled via digital pins of the Arduino, enabling automatic adjustments to reduce congestion and improve environmental conditions.

While previous works have either focused on object detection or adaptive signal logic, this study tries to present an integrated system that combines both: object detection using YOLOv5 and signal timing adjustment via an Arduino-based simulation. The proposed pipeline includes dataset preprocessing, vehicle detection, green light duration calculation Python, and visualizing through SimulIDE. All of the related works are summarized in Table 1, demonstrating some of the studies and researches that has been carried out on dynamic control of traffic light signal using Arduino. This table provide insights such as method or technique used, conclusion (mentioned as Findings) and some limitations about each research paper mentioned in the table.

Table 1. Literature review summarized in a tabular form comparing them with this study

Title	Method	Findings	Limitations
Adaptive Traffic Control System using Reinforcement Learning Traffic Signal Control Methods: Current Status, Challenges, and Emerging Trends	Q- learning-based reinforcement learning agent	Makes optimum decision based on rewards	Needs a training phase and simulated environment
Densely-Populated Traffic Detection Using YOLOv5 and Non-Maximum Suppression Ensembling	Survey of different approaches to traffic signal control methods like GA, fuzzy logic, and CV based	Broad coverage of traditional and upcoming techniques	Lacks implementation detail and quantitative comparison
Machine Learning Solutions for Adaptive Traffic Signal Control: A Review of Image Based Approaches	Using YOLOv5 and NMS ensembling	Fast detection and mAP@0.5 of 0.458	Accuracy in sparse conditions
Smart Traffic Management of Vehicles Using Faster R-CNN Based Deep Learning Method	Reviewing the RCNN, Fast RCNN, Faster RCNN, SSD, YOLOv4 and YOLOv8 ; testing in SUMO simulation	YOLO models have more potential than the others	No real-word testing under different conditions
Smart Autonomous Traffic Light Switching by Traffic Density Measurement Through Sensors	FRCNN with background modeling used on COCO and DAWN dataset	Handles illumination and shadows well	Slow than single stage models like YOLO
Density-Based Traffic Control System Using IoT and Machine Learning	IR sensors, low power embedded controllers	Low cost, and enables traffic light to switch from red to green	Limited in adaptability and complex cases
	YOLO and Arduino Mega 2560	Prioritizes ambulances, and introduces DBTCS	Tested on single lane, and needs scalability

Arduino-Powered Traffic Density Monitoring and Control	PIR sensors and Arduino integration	Tries to have real-time control with minimal computing	Accuracy drops in high-density traffic
Intelligent Traffic Light System Based on IoT	Arduino, Ultrasonic sensors, air quality sensors	Measures traffic density and air pollution levels	Requires multiple sensor types and calibration
Adaptive Traffic Control using Deep Learning and Arduino Simulation	YOLOv5 and Arduino simulation	Compares the three models – FRCNN, SSD, and YOLO on inference time and integrates with Arduino simulation in SimulIDE	Tested on static images, and not yet deployed in real world

3 Methodology

The methodology for this study involves several key stages, including data collection, pre-processing, model implementation, and Arduino-based integration. Those are discussed in the below subsections and summarized in Fig. 1.

3.1 Data collection

The dataset used in the study was acquired from Kaggle and uploaded to Google Colab. Various Python libraries, including torch, cv2, os, PIL, and time are required for image processing and analysis. Some of the key features of the dataset are listed below:

- **Diverse Geographic Coverage:** This dataset comprises traffic camera images from multiple countries, offering a global perspective on traffic monitoring and management
- **High-Quality Annotations:** Every image is carefully labeled with bounding boxes to mark different entities, such as vehicles, pedestrians, and traffic signs. This makes the dataset highly suitable for object detection tasks.
- **Diverse Environmental Scenarios:** The dataset features images taken in various weather conditions, lighting situations, and traffic environments, enhancing its applicability to real-world use cases
- **Training-Ready:** The dataset used is effective in training model YOLOv5, achieving a mean average precision (mAP) of 0.89, with precision of 0.88 and recall of 0.89. This can be used directly in the project.
- **Open for Research and Innovation:** As shared by the Kaggle, this dataset promotes collaboration and drives advancements in computer vision and traffic management. It can be a beneficial resource for research and developers to create innovative solutions.

3.2 Image preprocessing

To enhance dataset diversity augmentation techniques are applied. It is done by applying a range of transformations to the image such as rotation, cropping, scaling, flipping, adjusting brightness or contrast, and introducing noise. Augmentation simulates real-world situations, hence enhancing the robustness of the model. This process also helps in improving the model's ability to generalize and reduce overfitting. The authors have also applied a transformation on the dataset, to increase generalization in the image dataset.

Resizing is another essential step in image pre-processing that involves adjusting the dimensions of images to fit the expected input size of the model. Resizing provides the consistency in image dimensions that is necessary for efficient model training. To avoid image distortion the aspect ratio is often preserved. Together, augmentation and resizing are important to preparing image datasets for deep learning tasks.

3.3 Model implementation and evaluation

The authors implemented three models - YOLOv5, Faster R-CNN, and SSD- to analyze and perform the object detection task. These models examine traffic images captured by CCTV cameras to estimate vehicle density or count. Based on this analysis, they calculate the optimal green light duration. We tried to discuss the three models in the following paragraphs.

YOLOv5 (You Only Look Once version 5), introduced via GitHub is a neural network learning model designed for detecting objects. It works by segmenting an image given as an input into a grid then predicting class probabilities and bounding boxes for objects within every grid cell. In the initial step, feature extraction is done, where convolutional layers detect patterns and features in the image, such as edges and shapes. These features are passed through a series of layers that refine detections and predict bounding boxes, class probabilities, and object confidence. YOLOv5 employs anchor boxes to handle objects of different scales and shapes effectively. Due to its architecture and advanced techniques like auto-learning anchors and mosaic data augmentation, it is faster and more efficient. It is generally used in applications that require accuracy and fast object detection.

Faster R-CNN, introduced by Ren et al. [1] is a pioneer two-stage model that is an improved version to detect objects. Due to the introduction of the Region Proposal Network (RPN). The process begins with a backbone network, typically a Convolutional Neural Network, which uses the given input image to extract a feature map. Then RPN produces region proposals by sliding a small window across the feature map to predict potential object regions. These proposals are refined to improve accuracy. The regions of interest (RoI) are passed through the RoI pooling layer. It is done to normalize their size and to make them ready for bounding box regression and classification. The final step involves classifying the objects within the proposed regions and giving the bounding boxes for precise localization. Faster R-CNN is highly accurate, but generally slower than models like YOLO, as it is a two-stage process. It can be suitable to be used in applications requiring detailed object detection.

SSD (Single Shot Detector) is also a model to detect objects in a single pass via a network, introduced by Liu et al. [2]. It is faster than two-stage models like Faster R-CNN. It utilizes a convolutional neural network for the prediction of object classes and bounding box locations at multiple feature scales, enabling the detection of objects of varying sizes. SSD works by first extracting feature maps from an input image through a CNN, like VGG16 or ResNet. Then these feature maps are passed through several convolutional layers, each with a different level of abstraction. At each layer, SSD generates multiple bounding boxes with various scales and aspect ratios, which are refined to fit the objects in the image. For each box, SSD predicts the class of the object and adjusts the box coordinates. During training, the model learns to match ground truth objects to the most appropriate default boxes. The model then outputs the final detected objects by selecting boxes with high confidence scores. SSD is known for its efficiency and speed, making this ideal for real-time tasks for detecting objects, although it may not be as accurate as two-stage object detectors.

The authors compared these three models based on inference time, which refers to the time taken by each model to process the entire dataset and determine the green light duration mentioned in the studies by Qadri et al. [14] and Gorodokin et al. [15]. Since the dataset is split into training, testing, and validation sets, authors have calculated the time taken by each model for each set and observed which model took the least time for a particular set.

3.4 Model selection

After comparison based on inference time between the three models - YOLOv5, Faster R-CNN, and SSD, it was observed that YOLOv5 has the fastest processing speed along with higher accuracy. Due to this observation, the authors went forward with this model and did further implementation. This approach was used for processing the images and estimating the vehicle count, which serves the purpose of developing a dynamic traffic system.

3.5 Integration with Arduino

Once the vehicle count was estimated by processing the images, it was transmitted to an Arduino-based system via serial communication. To establish this serial communication Virtual Serial Port Emulator was used to map two different COM ports. Authors have used COM port 4 and COM port 5 to establish this communication. COM port 4 was used in Python code and COM port 5 was used in Arduino-based simulation. The Arduino code then used this vehicle count to calculate the optimal green light duration, ensuring that traffic signals adapt to real-time traffic density. The green light duration was calculated based on the following steps:

- minGreenTime parameter ensures that when the traffic is low that is vehicle count is less than five (threshold value) then the green light will be on for 10 seconds.
- maxGreenTime limits the duration to prevent excessive waiting time for other lanes which is 30 seconds.

- If the vehicle count exceeds the threshold value (5 vehicles) the duration is dynamically calculated by the following formula:

$$GreenLightDuration = \min(minGreenTime + (vehicleCount * 2), maxGreenTime) \quad (1)$$

The $\min()$ function ensures that the duration does not exceed v $maxGreenTime$ (30 seconds), even if the vehicle count is very high. This calculated green light duration is sent to the Arduino setup in SimulIDE, where the green light is switched on for the calculated duration.

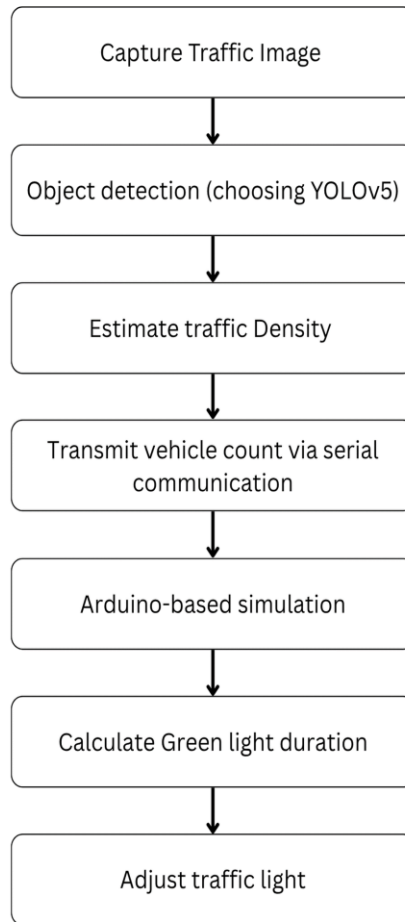


Fig. 1. Flowchart of used methodology

4 Results and Discussion

This section compares the performances of these three widely used models: YOLOv5 (You Only Look Once), Faster RCNN (Region-based Convolutional Neural Network), and SSD (Single Shot MultiBox Detector) in this study. Fig.2 shows a sample image from the test image set, its augmented and resized forms that are different for each model, and lastly the inferences from the YOLOv5, Faster R-CNN, and SSD models, respectively. In addition to the evaluation of model performance, this section also discusses the integration into the Arduino environment and its outcomes.

The authors applied the augmentation to bring diversity and resized the images according to the input requirements of each model. All the images are resized to match the input format required by each model, 416x416 for YOLO, 800x800 for Faster R-CNN, and 300x300 for SSD. Each image was rotated by 15 degrees and horizontally flipped to introduce augmentation to the dataset. This was done to ensure a variable environment for these models. A sample from the traffic dataset is shown in the Fig. 2, the top left image shows the original. The top right image presents the object detection results from the YOLOv5. The bottom left and bottom right displays the output of the Faster RCNN and SSD, respectively.

The green light duration was formulated on the basis of vehicle density, by counting the vehicles in each image. Vehicle density is calculated by counting the number of vehicles identified through object detection. When the total of vehicles is below a specified value point, the green light duration is kept at a minimum value. However, as the vehicle count increases beyond this threshold, the green light duration increases proportionally, up to a maximum limit. This approach ensures that traffic lights remain green longer when there is heavy traffic, improving traffic flow.

These models were judged based on the inference times as mentioned in Table 2 and Table 3. Table 2 shows the inference times before augmentation, while Table 3 presents the inference times after augmentation. Hence concluded that YOLOv5 showed an inference time of 87.49 (rounded-off) for the training set, 4.75 for the testing set, and 7.57 for the valid set. Faster R-CNN showed an inference time of 344.91 for the training set, 16.51 for the testing set, and 32.40 for the valid set. SSD showed an inference time of 236.96 for the training set, 11.35 for the testing set, and 22.66 for the valid set. It can be seen that SSD is faster than R-CNN, and YOLOv5 is faster than both of the other models. However, the augmentation and resizing make the datasets too randomized which changes the final inference time. However, these randomizations do not change the fact that YOLOv5 outperforms the other two models, which is also backed by a study by Mehta et al. [8].

Table 2. Time taken by the three models, in each set of datasets

Models	Training Set	Testing Set	Valid Set
YOLOv5	72.59238958358765	3.213449239730835	6.800236701965332
Faster R-CNN	345.46866250038147	16.72565531730652	32.54366064071655
SSD	232.3265721797943	11.15900206565857	219.03977632522583

Table 3. Time taken by the after augmentation and resizing

Models	Training Set	Testing Set	Valid Set
YOLOv5	87.48702955245972	4.754430294036865	7.5696046352386475
Faster R-CNN	344.91410779953003	16.51194977760315	32.4020357131958
SSD	236.95764017105103	11.349228382110596	22.663291215896606

The integration of the detection model with the traffic light controller was done using SimulIDE and an Arduino environment. It was working in a way such that it took input from the Python code using SimulIDE. The system worked reliably as the green light was in an ON state, according to the time calculated by the code using the images. It helped us visualize how green lights could be implemented in real-time implementation, that however is a future scope in our work that would help in reducing the traffic around the peak hours.

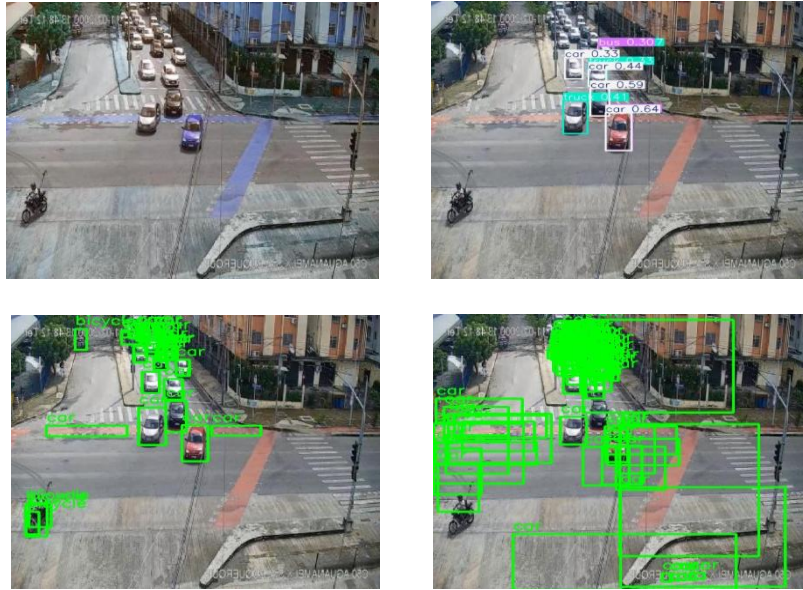


Fig. 2. Left to right from top – Original image from the traffic dataset, object detection by YOLOv5, object detection by FRCNN, and object detection by SSD.

5 Conclusion

In this study, the authors analyzed a traffic image dataset using artificial intelligence techniques. Three deep-learning models were utilized for the analysis: YOLOv5, Faster

R-CNN, and SSD. These models were designed to detect objects in real time, allowing them to process images and calculate the total of vehicles. Based on vehicle count, the green light duration is dynamically calculated, reducing unnecessary waiting times at traffic signals. While many existing research studies such as those done by Rabiou et al. [16] focus primarily on reducing CO₂ emissions and improving air quality as in the study by Gupta et al. [13], this work emphasizes minimizing wait times and comparing the performance of these three models to identify the most efficient one for traffic control.

The findings indicate that YOLOv5 consistently takes the least amount of time to process the entire dataset and calculate the green light duration for all images. One limitation of this study is that image augmentation and resizing involve randomized functions, causing variations in inference time. However, it was consistently observed that YOLOv5 outperforms the other models in terms of processing speed. Therefore, these variations do not alter the conclusion that YOLOv5 is the most efficient model for this task.

Based on these findings, the authors ultimately selected YOLOv5 for vehicle count estimation and successfully integrated it with an Arduino-based simulation, following approaches similar to Tabassum et al. [11]. The vehicle count obtained from the Python-based detection system was transmitted to the Arduino code using serial communication, where the optimal green light duration was computed. This calculated duration was then sent to an Arduino simulation in SimulIDE, ensuring that the green light remained active for the appropriate time based on real-time traffic density.

This work has successfully integrated deep learning models and an Arduino environment to yield a functional traffic control system. However, the prototype is limited to single-junction simulations and static images from a dataset, it can be extended in the direction of the real-time video processing from live traffic cameras.

References

1. Ren, S., He, K., Girshick, R., Sun, J.: Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39(6), 1137–1149 (2017)
2. Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.Y., Berg, A.C.: Ssd: Single shot multibox detector. In: Leibe, B., Matas, J., Sebe, N., Welling, M. (eds.) *Computer Vision – ECCV 2016*. pp. 21–37. Springer International Publishing, Cham (2016)
3. Khan, I., Hasan, M., Islam, S., Reza, S., Reza, T., Efaz, E.T.: Adaptive traffic signal control system using object detection based approach. In: *2025 4th International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST)*. pp. 86–91 (2025)
4. Shingate, K., Jagdale, K., Dias, Y.: Adaptive traffic control system using reinforcement learning. *International Journal of Engineering Research and V9* (02 2020)
5. Tomar, I., Indu, S., Pandey, N.: Traffic signal control methods: Current status, challenges, and emerging trends. In: Gupta, D., Polkowski, Z., Khanna, A., Bhattacharyya, S., Castillo, O. (eds.) *Proceedings of Data Analytics and Management*. pp. 151–163. Springer Nature Singapore, Singapore (2022)

6. Khasawneh, M.A., Awasthi, A.: Intelligent meta-heuristic-based optimization of traffic light timing using artificial intelligence techniques. *Electronics* 12(24) (2023), <https://www.mdpi.com/2079-9292/12/24/4968>
7. Rahman, R., Bin Azad, Z., Bakhtiar Hasan, M.: Densely-Populated Traffic Detection Using YOLOv5 and Non-maximum Suppression Ensembling, p. 567–578. Springer Singapore (Dec 2021), http://dx.doi.org/10.1007/978-981-16-6636-0_43
8. Mehta, K., Raj, K., Brahmbhatt, K.: Machine learning solutions for adaptive traffic signal control: A review of image-based approaches. *World Journal of Advanced Engineering Technology and Sciences* 13, 476–481 (09 2024)
9. Chaudhuri, A.: Smart traffic management of vehicles using faster r-cnn based deep learning method (2023), <https://arxiv.org/abs/2311.10099>
10. Jagadeesh, Y.M., Suba, G.M., Karthik, S., Yokesh, K.: Smart autonomous traffic light switching by traffic density measurement through sensors. In: 2015 International Conference on Computers, Communications, and Systems (ICCCS). pp. 123–126 (2015)
11. Tabassum, H., Mohammed Ameer, M., Alam, M.M., Sharieff, M.Y., Ahamed, N., Anjum, A.: Density-based traffic control system using iot and machine learning. In: 2024 International Conference on Knowledge Engineering and Communication Systems (ICKECS). vol. 1, pp. 1–6 (2024)
12. Srivastava, P.C., Srivastava, K., Dixit, A., Singh, A., Singh, A.: Arduino-powered traffic density monitoring and control. In: 2023 3rd International Conference on Advancement in Electronics Communication Engineering (AECE). pp. 1024–1027 (2023)
13. Gupta, R., Singh, A.K., Tewari, P.P., Dabral, S.: Intelligent traffic light system based on iot. In: 2023 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS). pp. 1630–1635 (2023)
14. Qadri, S.S.S.M., Gökçe, M., Oner, E.: State-of-art review of traffic signal control methods: challenges and opportunities. *European Transport Research Review* 12, 1–23 (10 2020)
15. Gorodokin, V., Alferova, I., Shepeleva, E.: Calculating the duration of the traffic light green interval allowing pedestrians entering the traffic way. *Transportation Research Procedia* 36, 220–224 (01 2018)
16. Rabiou, H., Bashir, H.: Intelligent traffic light system for green traffic management (12 2015)